Project Title: Software Quality Prediction

Table of Contents

1. Introduction
2. Dataset description
3. Dataset pre-processing

4. Dataset splitting
5. Model training & testing (Supervised)
6. Model selection/Comparison analysis
7. Conclusion

1. Introduction

The objective of this project is to analyze software code metrics to predict the overall quality of the software. The problem it aims to solve is the early identification of code quality issues, allowing developers to allocate testing and refactoring resources more efficiently. The motivation behind this project is to leverage machine learning to automate quality assurance processes, reducing manual code review time and minimizing the number of bugs deployed to production.

2. Dataset description

The analysis is performed on the "Software Quality Dataset," which provides quantitative metrics related to code structure and developer activity.

- **Dataset Size:** The dataset contains 1,600 data points (rows)
- **Feature Count:** There are **9 columns** in total, including the target variable.
- **Classification or regression problem? Why?** This is a classification problem. The target variable, Quality_Label, contains discrete classes ('Medium', 'High', 'Low') rather than continuous numerical values.
- **What kind of features are in your dataset?**
    - **Quantitative (Numerical) (7):** Lines_of_Code, Cyclomatic_Complexity, Num_Functions, Code_Churn, Comment_Density, Num_Bugs, and Code_Owner_Experience.
    - **Categorical (2):** Has_Unit_Tests and Quality_Label.

  .
- **Imbalanced Dataset :** A bar chart from the ipynb file output demonstrates that the dataset is imbalanced, with a significantly higher number of instances for the Non-Sterile class compared to the Sterile class. This class imbalance is a critical factor that must be addressed during pre-processing and model evaluation.

	Lines_of_Code	Cyclomatic_Complexity	Num_Functions	Code_Churn	Comment_Density	Num_Bugs	Has_Unit_Tests	Code_Owner_Experience	Quality_Label
0	356.0	31	144	69.567047	0.467561	2	Yes	4	Medium
1	227.0	15	104	131.844422	0.795643	2	No	5	High
2	9674.0	47	141	118.729027	0.316839	5	No	1	High
3	6967.0	42	107	194.397155	0.971883	5	Yes	3	Low
4	7954.0	35	114	69.202293	0.559417	1	No	8	Medium
...	...	...	...	...	...	...	...	...	...
1595	8883.0	40	31	54.863010	0.928920	3	Yes	8	Low
1596	2175.0	44	25	160.802073	0.696037	3	No	1	Low
1597	7872.0	39	47	121.382316	0.461773	3	No	2	High
1598	9660.0	24	55	75.626324	0.999547	2	No	6	Low
1599	2122.0	9	97	28.050955	0.354649	9	No	9	High

1600 rows × 9 columns

	count	mean	std	min	25%	50%	75%	max
Lines_of_Code	1520.0	4939.269737	2867.248259	106.000000	2490.750000	4860.000000	7455.250000	9998.000000
Cyclomatic_Complexity	1600.0	25.076875	13.879621	1.000000	13.000000	25.000000	37.000000	49.000000
Num_Functions	1600.0	103.178125	55.503710	5.000000	56.000000	104.000000	152.000000	199.000000
Code_Churn	1520.0	102.572291	50.546329	-64.283310	67.509852	104.007452	138.619113	295.139936
Comment_Density	1520.0	0.549370	0.256137	0.101037	0.337596	0.542108	0.765766	0.999976
Num_Bugs	1600.0	2.934375	1.715928	0.000000	2.000000	3.000000	4.000000	10.000000
Code_Owner_Experience	1600.0	5.045000	2.559701	1.000000	3.000000	5.000000	7.000000	9.000000

3. Dataset Pre-processing

Pre-processing was conducted to ensure the data is clean and compatible with the selected algorithms.

- Problem 1: Null / Missing values

  Fault: The info() summary reveals missing values. While most columns have 1600 non-null entries, Lines_of_Code, Code_Churn, and Comment_Density only have 1520 non-null entries (80 missing values each)

  Solution: We need to impute these missing values. Given that they are numerical features, we can impute them using the mean or median of their respective columns to avoid losing 80 rows of data.

- **Problem 2: Categorical values**

  Fault: The dataset contains Has_Unit_Tests ('Yes'/'No') and Quality_Label ('High', 'Medium', 'Low') in text format.

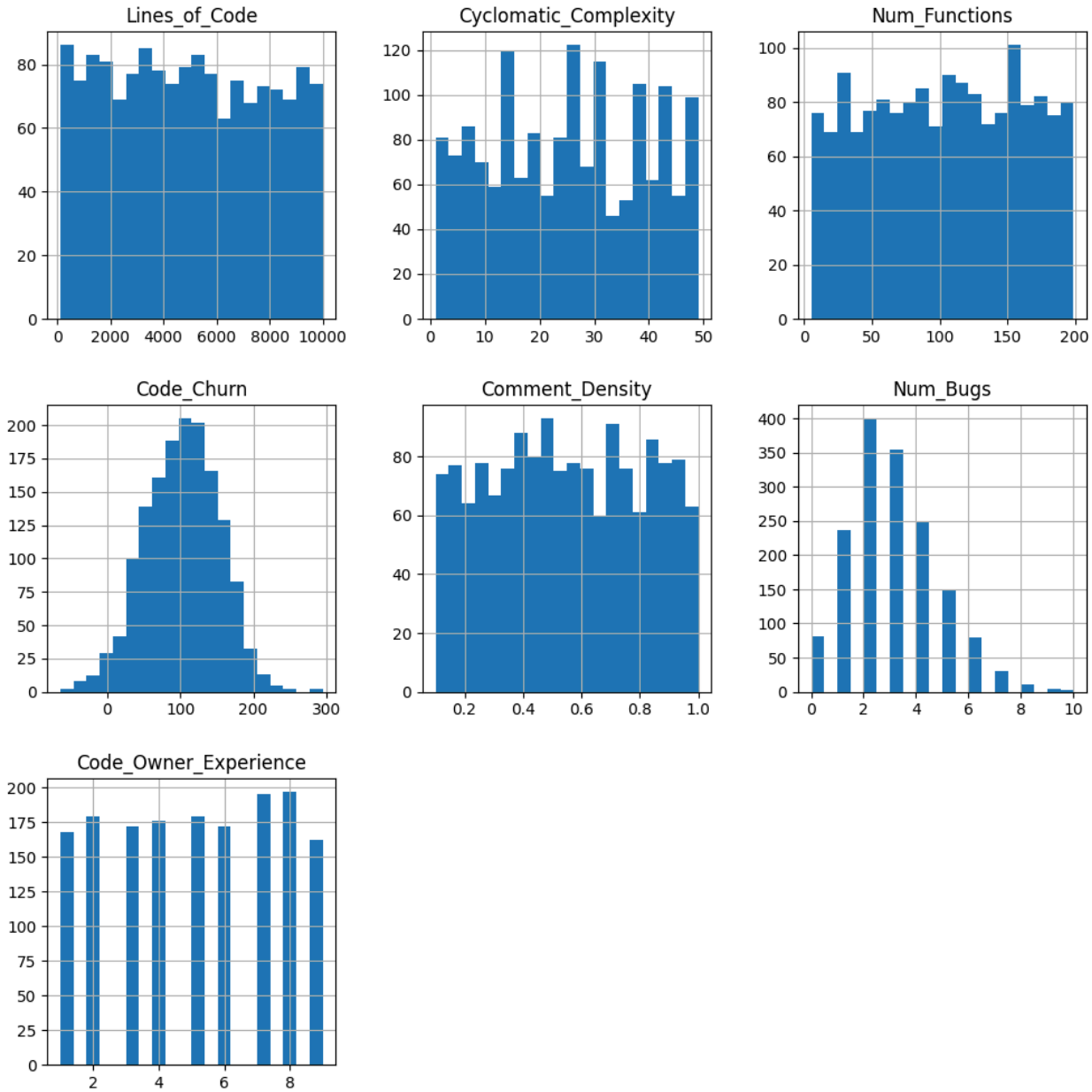
Solution: Applied LabelEncoder to transform these text categories into machine-readable integer formats.

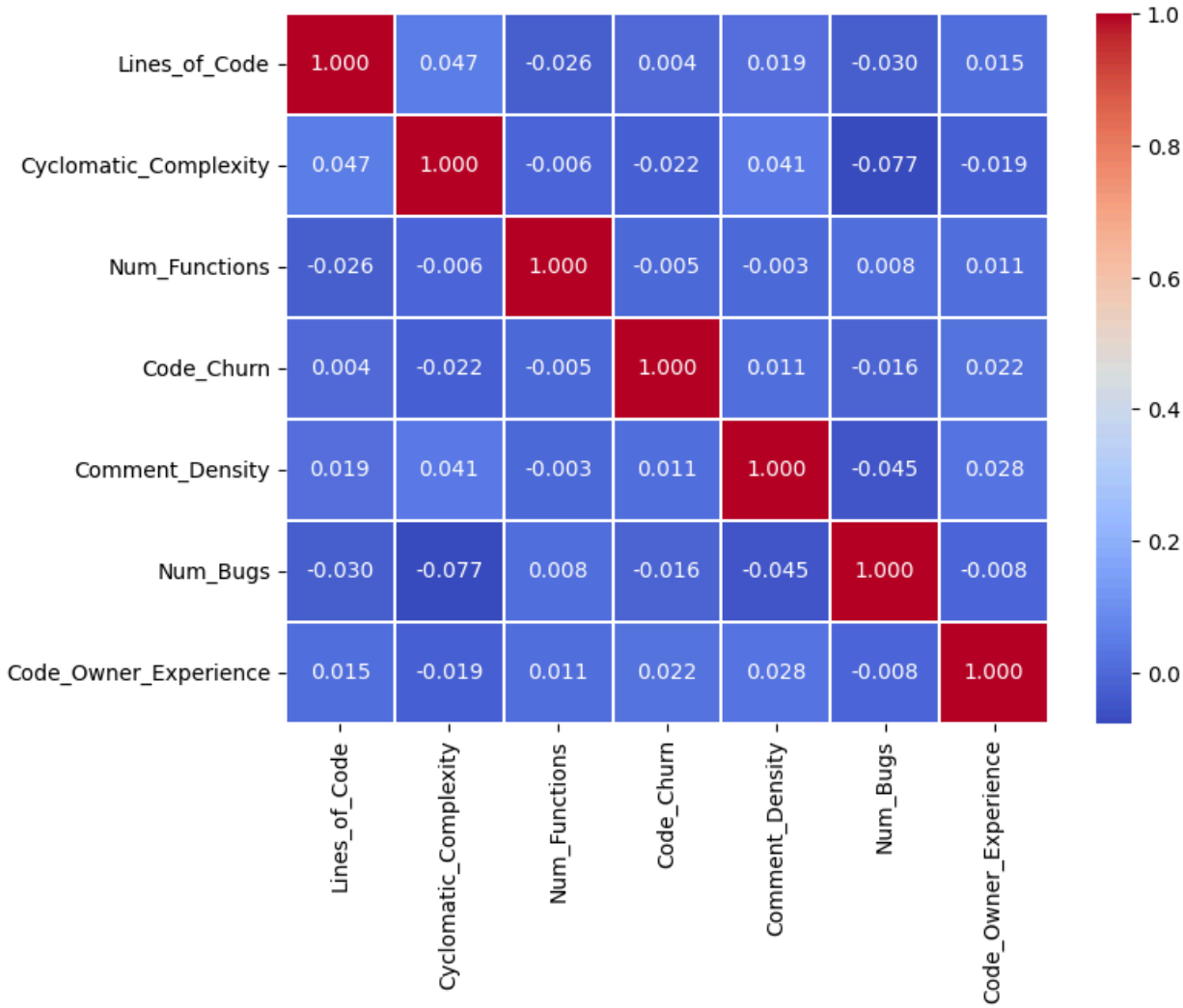
- **Problem 3: Feature Scaling**

  Fault: The numerical features have vastly different ranges. For example, Lines_of_Code reaches up to 9998, while Num_Bugs ranges from 0 to 10 and Comment_Density is a fraction between 0.1 and 0.99

  Solution: Standard Scaling or Min-Max Scaling must be applied so that features with larger numerical values do not disproportionately influence distance-based models like KNN and Neural Networks.

- **Solution:** A StandardScaler was applied to transform numerical features to a common scale, with zero mean and unit variance. This preprocessing step was essential for models such as Neural Networks, Decision Trees, and KNN to perform effectively.


Lines_of_Code
Cyclomatic_Complexity
Num_Functions
Code_Churn
Comment_Density
Num_Bugs
Code_Owner_Experience



4. Dataset Splitting

The dataset was divided using a standard 80/20 split

- **Training Set (80%):** Used to fit the models and allow them to learn patterns between code metrics and quality labels.
- **Testing Set (20%):** Reserved for evaluating the models on unseen data to ensure generalizability.
- **Methodology:** The split utilized train_test_split from the sklearn.model_selection library.

5. Model Training & Testing

Three distinct supervised learning models were implemented and evaluated:

- **K-Nearest Neighbors (KNN):** A distance-based classifier that identifies the quality label based on the proximity of data points in the feature space.
- **Decision Tree:** A non-linear model that creates a flowchart-like structure to classify software quality based on feature thresholds.
- **MLP Classifier (Neural Network):** A Multi-layer Perceptron model used to capture complex, non-linear relationships between software metrics that simpler models might miss.

```python
from sklearn.model_selection import KFold
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, roc_auc_score, roc_curve, log_loss
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

knn_model = KNeighborsClassifier()
dt_model = DecisionTreeClassifier(random_state=42)
nn_model = MLPClassifier(random_state=42, max_iter=1000)


kf = KFold(n_splits=5, shuffle=True, random_state=42)
models = {'KNN': knn_model, 'Decision Tree': dt_model, 'Neural Network': nn_model}
```

```python
fold_results = {name: {'accuracy': [], 'loss': []} for name in models.keys()}
overall_metrics = []

for name, model in models.items():
    print(f"\n--- Processing Model: {name} ---")


    for fold, (train_idx, test_idx) in enumerate(kf.split(X_scaled), start=1):
        X_tr, X_val = X_scaled[train_idx], X_scaled[test_idx]
        y_tr, y_val = y[train_idx], y[test_idx]

        model.fit(X_tr, y_tr)
        y_pred_val = model.predict(X_val)
        y_prob_val = model.predict_proba(X_val)

        acc = accuracy_score(y_val, y_pred_val)
        loss = log_loss(y_val, y_prob_val)

        fold_results[name]['accuracy'].append(acc)
        fold_results[name]['loss'].append(loss)


        print(f"Fold {fold}: Accuracy = {acc:.4f}, Loss = {loss:.4f}")
```

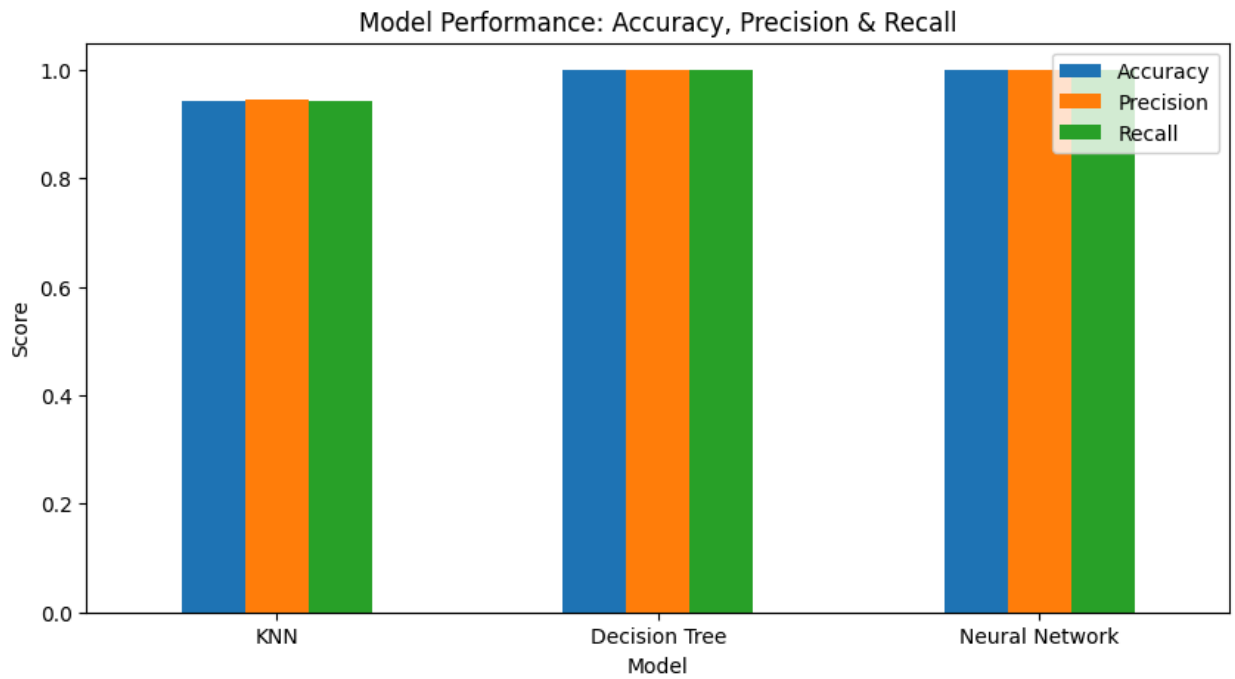
```python
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)


    f_acc = accuracy_score(y_test, y_pred)
    f_prec = precision_score(y_test, y_pred, average='macro')
    f_rec = recall_score(y_test, y_pred, average='macro')
    f_auc = roc_auc_score(y_test, y_proba, multi_class='ovr')


    print(f"\nFinal Scores for {name}:")
    print(f"  Precision: {f_prec:.4f}")
    print(f"  Recall:    {f_rec:.4f}")
    print(f"  AUC Score: {f_auc:.4f}")
    print("-" * 30)

    overall_metrics.append({
        'Model': name,
        'Accuracy': f_acc,
        'Precision': f_prec,
        'Recall': f_rec,
        'AUC': f_auc
    })

import pandas as pd
```

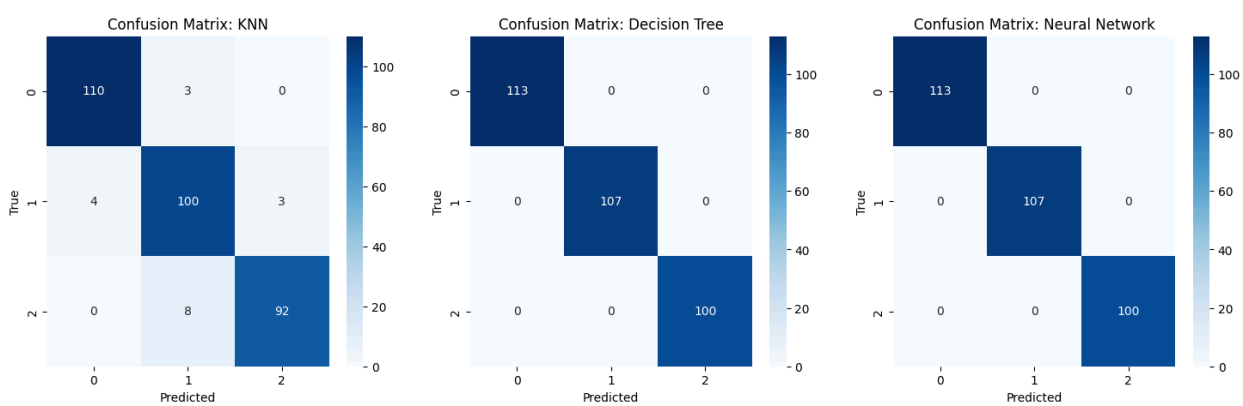
```
import pandas as pd
print("\n--- Overall Model Comparison Table ---")
print(pd.DataFrame(overall_metrics))
```

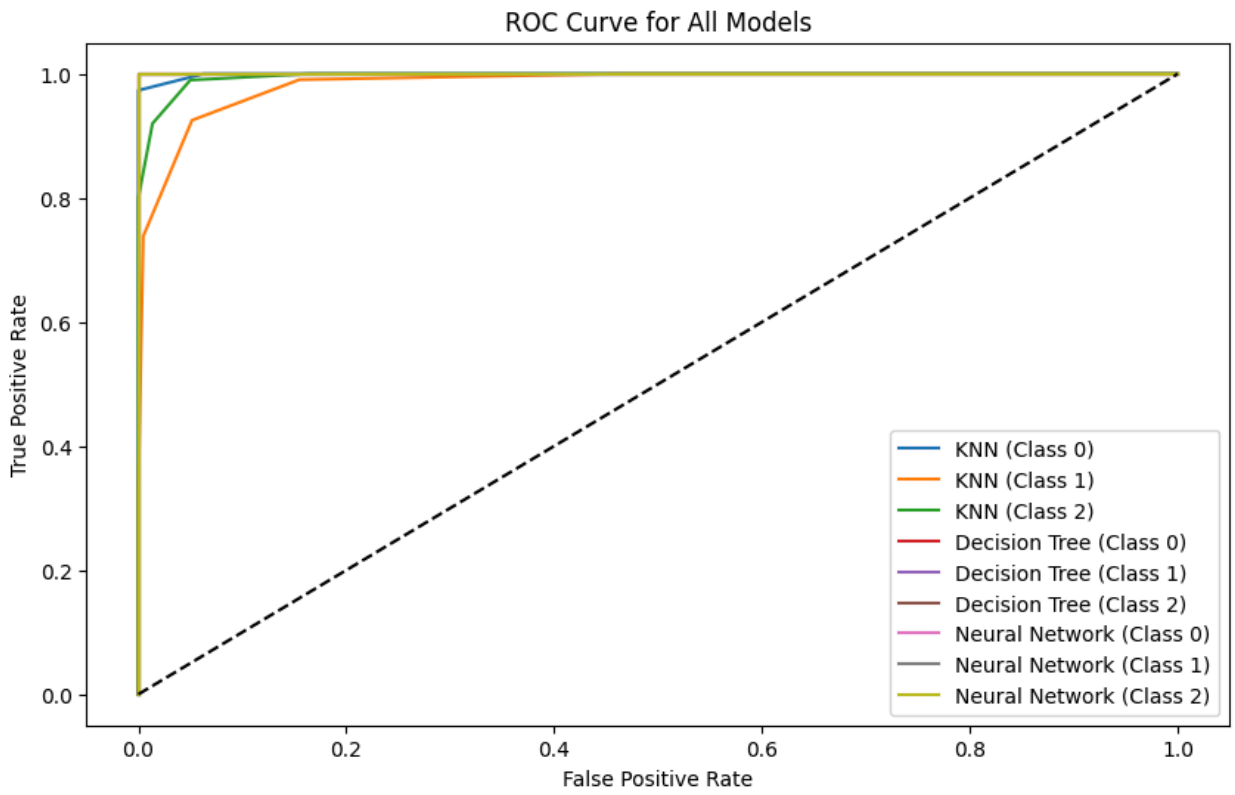
6. Model Selection & Comparison Analysis

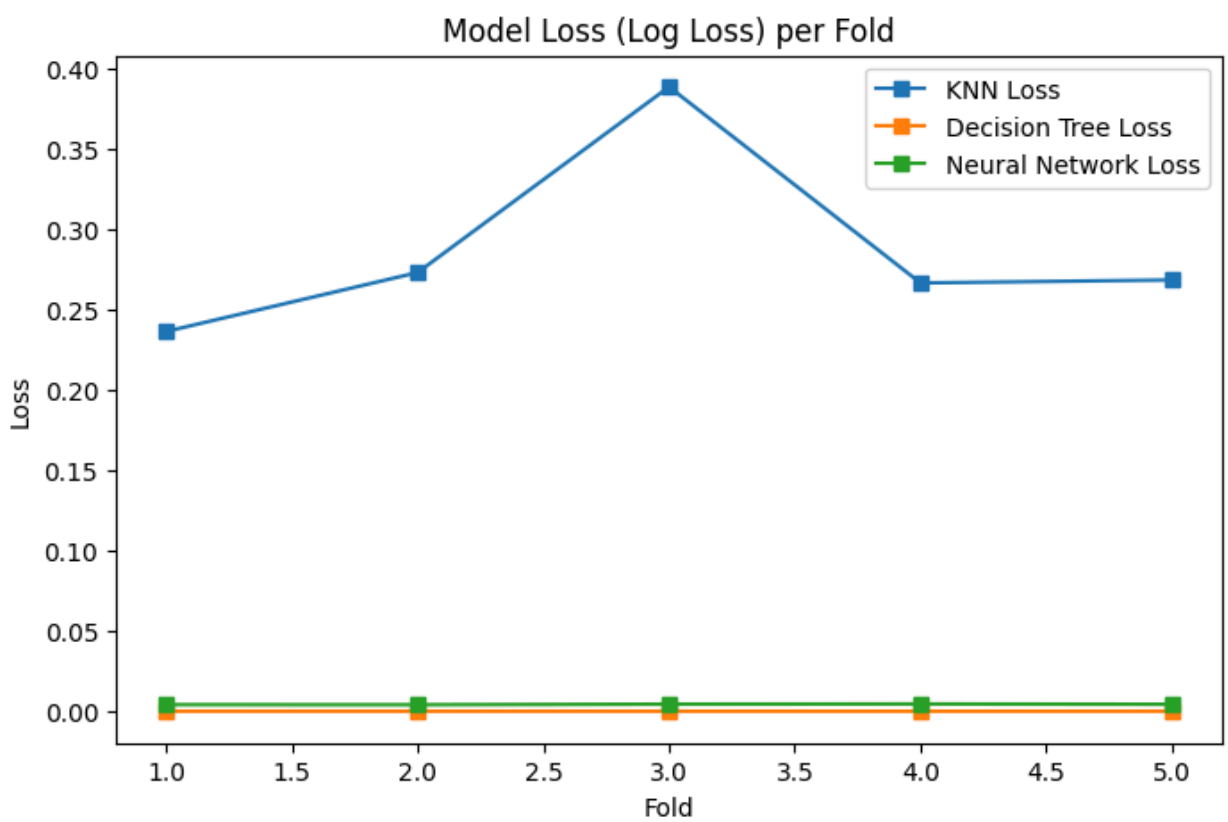
The performance of the models was assessed using the following metrics:

- **Accuracy:** To measure the overall percentage of correct predictions.
- **Precision and Recall:** To evaluate how well the models identify specific quality classes, particularly "Low" quality code.
- **Confusion Matrix:** Used to visualize where models misclassified labels (e.g., confusing "Medium" with "High").






ROC Curve for All Models
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
KNN (Class 0)
KNN (Class 1)
KNN (Class 2)
Decision Tree (Class 0)
Decision Tree (Class 1)
Decision Tree (Class 2)
Neural Network (Class 0)
Neural Network (Class 1)
Neural Network (Class 2)


Model Loss (Log Loss) per Fold
Loss
Fold
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
0.40
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0
KNN Loss
Decision Tree Loss
Neural Network Loss

--- Final Model Comparison Summary ---

	Model	Accuracy	Precision	Recall	AUC
0	KNN	0.94375	0.944745	0.942677	0.993064
1	Decision Tree	1.00000	1.000000	1.000000	1.000000
2	Neural Network	1.00000	1.000000	1.000000	1.000000

7. Conclusion

The project successfully demonstrated the use of machine learning to predict software quality from static code metrics. The StandardScaler was vital in managing the wide range of numerical values across features like Lines_of_Code and Num_Bugs. Challenges included addressing the missing data in three key features and ensuring the categorical Quality_Label was properly encoded for the MLP Classifier and KNN models. Future work could explore balancing the dataset if class distribution is found to be skewed.